Development And Tooling Guide

This guide explains clone-time setup, local development checks, and model-maintenance tools.

Directory Roles

```
src/      operational conversion scripts
tools/    setup, model maintenance, taxonomy generation, and test artifact
helpers
tests/    regression checks
specs/    committed LHM, binding, currency, and model definition CSVs
samples/  committed sample XML and expected outputs
out/      local generated output, ignored by Git
```

The main invoice conversion runtime is in **src/**. Use **tools/** for setup, regeneration, test artifact building, taxonomy output, and specification maintenance.

Initial Setup After Clone

Windows PowerShell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd .\UADC-PoC
$python = 'python'
& $python --version
```

$python = 'python' is a PowerShell variable assignment used by examples. It means run the Python executable available on **PATH**. If Python is not on **PATH**, set it to the full executable path:

```
$python = 'C:\Users\
<user>\AppData\Local\Programs\Python\Python310\python.exe'
& $python --version
```

macOS or Linux shell:

```
git clone https://github.com/pontsoleil/UADC-PoC.git
cd ./UADC-PoC
PYTHON=python3
$PYTHON --version
```

Use **$PYTHON script.py** on macOS/Linux. Use **& $python script.py** only in Windows PowerShell.

Compile Core Scripts

Compile before running broad checks:

```
& $python -m py_compile `
  .\src\syntax_binding.py `
  .\src\syntax_binding_ads_xbrl_gl.py `
  .\src\semantic_binding.py `
  .\tools\build_roundtrip_test_artifacts.py `
  .\tools\taxonomy\xBRLGL_TaxonomyGenerator.py
```

macOS/Linux equivalent:

```
$PYTHON -m py_compile \
  ./src/syntax_binding.py \
  ./src/syntax_binding_ads_xbrl_gl.py \
  ./src/semantic_binding.py \
  ./tools/build_roundtrip_test_artifacts.py \
  ./tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Tool Classification

GitHub and development environment support:

```
build_roundtrip_test_artifacts.py
psv_viewer.html
```

Conversion-target model environment support:

```
taxonomy/xBRLGL_TaxonomyGenerator.py
build_lhm_from_source.py
build_syntax_binding.py
normalize_lhm_semantic_paths.py
normalize_lhm_class_element.py
check_lhm_class_element.py
audit_en16931_coverage.py
extend_en16931_lhm_coverage.py
order_lhm_by_en16931_table2.py
update_lhm_definitions_from_pdf.py
update_lhm_syntax_sequence_from_ubl_xsd.py
```

Tutorial and sample converters:

```
syntax_binding_sample.py
semantic_binding.py
```

Operational runtime converters are in **src/**, not **tools/**.

Model Inputs And Generated Outputs

Committed source definitions:

```
specs/lhm/EN16931_CIUS_Invoice_LHM.csv
specs/bindings/syntax/
specs/bindings/semantic/
specs/Currency.csv
```

Generated local outputs:

```
out/taxonomy/
out/phase1/
out/phase2/
tests/roundtrip/
```

out/ is local generated output and is normally ignored by Git.

Taxonomy Generation

Primary script:

```
tools/taxonomy/xBRLGL_TaxonomyGenerator.py
```

Role:

```
LHM CSV -> local xBRL-CSV taxonomy for Structured CSV metadata
```

Generate and validate through regression checks:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
```

Expected local output includes:

```
out/taxonomy/en16931/en16931-2026-07-05.xsd
out/taxonomy/gen/gl-gen-2026-07-05.xsd
out/taxonomy/plt/plt-oim-2026-07-05.xsd
out/taxonomy/plt/plt-def-2026-07-05.xml
```

src/syntax_binding.py does not generate taxonomy files. It references them through **--taxonomy-base** when writing xBRL-CSV metadata JSON.

LHM Maintenance

Scripts:

```
build_lhm_from_source.py
normalize_lhm_semantic_paths.py
normalize_lhm_class_element.py
check_lhm_class_element.py
audit_en16931_coverage.py
extend_en16931_lhm_coverage.py
order_lhm_by_en16931_table2.py
update_lhm_definitions_from_pdf.py
update_lhm_syntax_sequence_from_ubl_xsd.py
```

Typical checks after LHM edits:

```
& $python .\tests\test_lhm_semantic_paths.py
& $python .\tests\test_lhm_hierarchical_csv_layout.py
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tests\validate_taxonomy.py
```

Binding Maintenance

Syntax binding CSVs live under:

```
specs/bindings/syntax/
```

Operational scripts:

```
src/syntax_binding.py
src/syntax_binding_ads_xbrl_gl.py
```

After syntax binding changes:

```
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

Semantic binding CSVs live under:

```
specs/bindings/semantic/
```

Operational script:

```
src/semantic_binding.py
```

Semantic binding tables start from a target table definition and add **semantic_path**, **type**, and **multiplicity**. The converter derives source columns from **semantic_path** and row scope from **type=C** rows in the binding table.

After semantic binding changes:

```
& $python .\tests\test_ads_invoices_received_psv.py
& $python .\tests\test_ads_invoices_generated_psv.py
& $python .\tests\test_phase2_outputs_by_structured_csv_stem.py
```

Round-Trip Artifact Regeneration

When LHM, syntax binding, or taxonomy metadata behavior changes:

```
& $python .\tools\build_roundtrip_test_artifacts.py
& $python .\tests\test_roundtrip_artifacts.py
```

Artifacts show:

```
source XML
Structured CSV
xBRL-CSV metadata JSON
regenerated UBL XML
```

Continuing Development Workflow

Before editing:

```
git status --short
```

After editing runtime scripts:

```
& $python -m py_compile .\src\syntax_binding.py
& $python .\tests\test_syntax_binding.py
& $python .\tests\test_syntax_binding_reverse.py
```

After editing LHM or binding CSV files:

```
& $python .\tests\test_xbrlgl_generator_uadc_lhm.py
& $python .\tools\build_roundtrip_test_artifacts.py
& $python .\tests\test_roundtrip_artifacts.py
```

Before sharing changes, run a broad local check and report clearly if Arelle or the UBL schema cache is not available.

psv_viewer.html

tools/psv_viewer.html displays generated ADS PSV files as a browser table. It runs entirely in the browser and does not require a local web server. It supports pipe, comma, and tab delimiters, row filtering, sticky headers, horizontal scrolling, and automatic hiding of columns that are empty in every data row.